

Лабораторна робота №5
Реалізація спроможностей реєстрації, аутентифікації та авторизації
засобами фреймворку ASP.NET Core Identity.

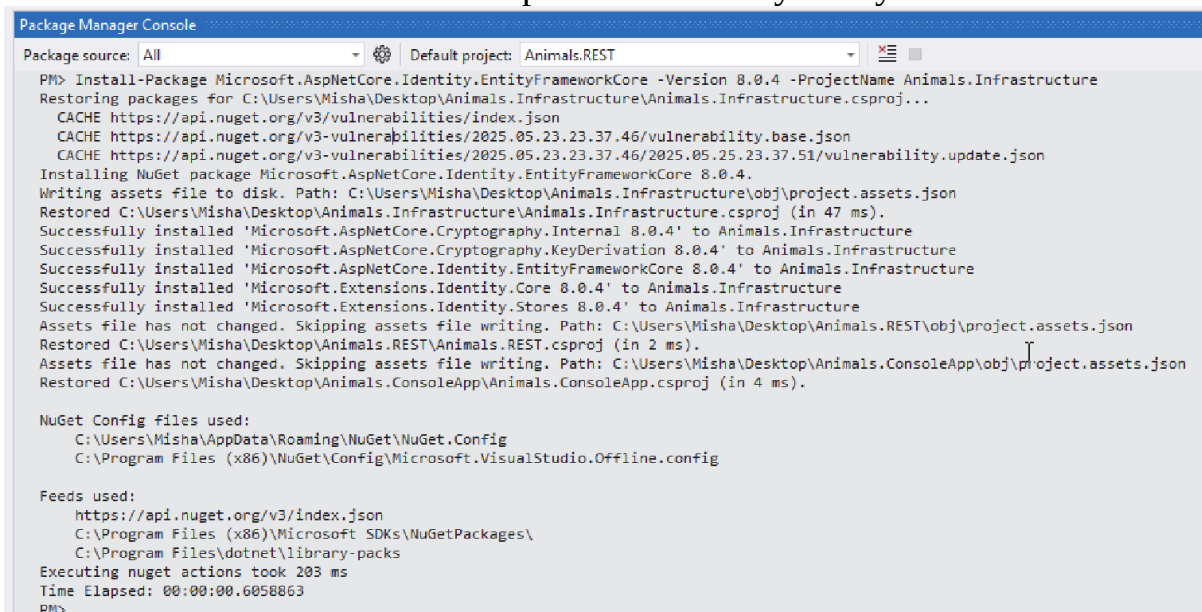
Виконав студент групи 302-ТК

Рогочий Михайло

Передумова:

Завдання

1. Додати в проєкт {Назва тематики}.Infrastructure нову залежність на NuGET пакет Microsoft.AspNetCore.Identity.EntityFrameworkCore.



```
Package Manager Console
Package source: All Default project: Animals.REST
PM> Install-Package Microsoft.AspNetCore.Identity.EntityFrameworkCore -Version 8.0.4 -ProjectName Animals.Infrastructure
Restoring packages for C:\Users\Misha\Desktop\Animals.Infrastructure\Animals.Infrastructure.csproj...
CACHE https://api.nuget.org/v3/vulnerabilities/index.json
CACHE https://api.nuget.org/v3-vulnerabilities/2025.05.23.23.37.46/vulnerability.base.json
CACHE https://api.nuget.org/v3-vulnerabilities/2025.05.23.23.37.46/2025.05.25.23.37.51/vulnerability.update.json
Installing NuGet package Microsoft.AspNetCore.Identity.EntityFrameworkCore 8.0.4.
Writing assets file to disk. Path: C:\Users\Misha\Desktop\Animals.Infrastructure\obj\project.assets.json
Restored C:\Users\Misha\Desktop\Animals.Infrastructure\Animals.Infrastructure.csproj (in 47 ms).
Successfully installed 'Microsoft.AspNetCore.Cryptography.Internal 8.0.4' to Animals.Infrastructure
Successfully installed 'Microsoft.AspNetCore.Cryptography.KeyDerivation 8.0.4' to Animals.Infrastructure
Successfully installed 'Microsoft.AspNetCore.Identity.EntityFrameworkCore 8.0.4' to Animals.Infrastructure
Successfully installed 'Microsoft.Extensions.Identity.Core 8.0.4' to Animals.Infrastructure
Successfully installed 'Microsoft.Extensions.Identity.Stores 8.0.4' to Animals.Infrastructure
Assets file has not changed. Skipping assets file writing. Path: C:\Users\Misha\Desktop\Animals.REST\obj\project.assets.json
Restored C:\Users\Misha\Desktop\Animals.REST\Animals.REST.csproj (in 2 ms).
Assets file has not changed. Skipping assets file writing. Path: C:\Users\Misha\Desktop\Animals.ConsoleApp\obj\project.assets.json
Restored C:\Users\Misha\Desktop\Animals.ConsoleApp\Animals.ConsoleApp.csproj (in 4 ms).

NuGet Config files used:
  C:\Users\Misha\AppData\Roaming\NuGet\NuGet.Config
  C:\Program Files (x86)\NuGet\Config\Microsoft.VisualStudio.Offline.config

Feeds used:
  https://api.nuget.org/v3/index.json
  C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\
  C:\Program Files\dotnet\library-packs

Executing nuget actions took 203 ms
Time Elapsed: 00:00:00.6058863
PM>
```

Встановив через консоль
Файл Animals.Infrastructure.csproj

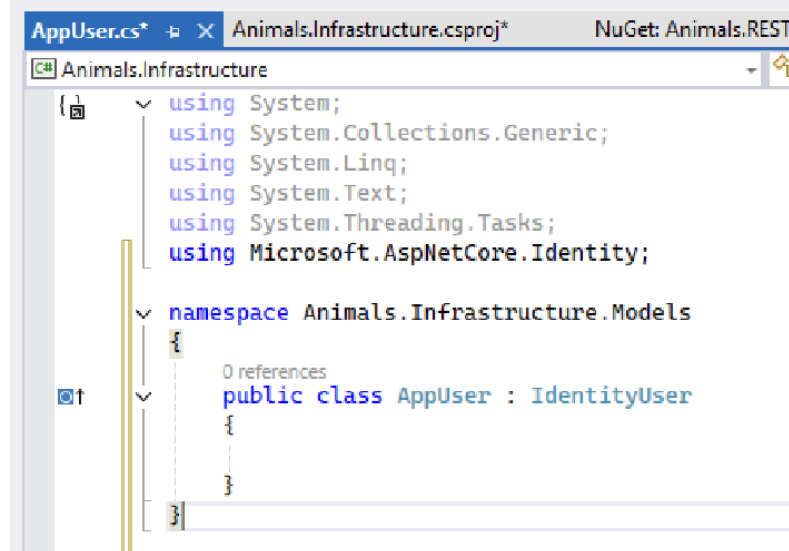


```
Animals.Infrastructure.csproj  x  NuGet: Animals.REST  NuGet: Animals.Infrastructure  EnclosureModel.cs  CrudServiceAsync.cs
<?xml version='1.0' encoding='utf-8'>
  <Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
      <TargetFramework>net8.0</TargetFramework>
      <ImplicitUsings>enable</ImplicitUsings>
      <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
      <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="8.0.4" />
      <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.4" />
      <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="9.0.4" />
    </ItemGroup>
  </Project>
```

2. Модифікуйте створену вами у попередніх лабораторних роботах модель (набір класів), додавши до неї сутність користувача, яка буде наслідуватися від класу IdentityUser із фреймворку ASP.NET Core Identity. Модифікуйте, створений у третій лабораторній, DbContext

таким чином, щоб він наслідувався від класу `IdentityDbContext<T>`, де узагальнюючим типом буде створена вами сутність користувача.

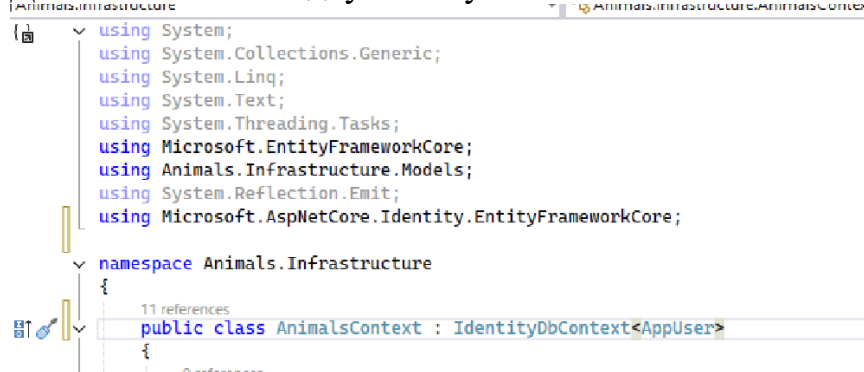
Створюємо новий клас `AppUser.cs` у проєкті `Animals.Infrastructure`



```
AppUser.cs*  Animals.Infrastructure.csproj*  NuGet: Animals.REST
Animals.Infrastructure
{
    using System;
    using System.Collections.Generic;
    using System.Linq;
    using System.Text;
    using System.Threading.Tasks;
    using Microsoft.AspNetCore.Identity;

    namespace Animals.Infrastructure.Models
    {
        0 references
        public class AppUser : IdentityUser
        {
        }
    }
}
```

Далі змінюємо спадкування у `AnimalContext`

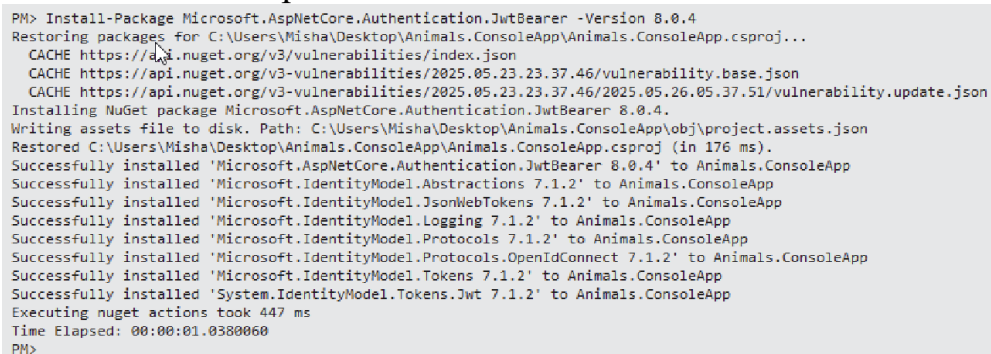


```
Animals.Infrastructure  Animals.Infrastructure/AnimalContext
{
    using System;
    using System.Collections.Generic;
    using System.Linq;
    using System.Text;
    using System.Threading.Tasks;
    using Microsoft.EntityFrameworkCore;
    using Animals.Infrastructure.Models;
    using System.Reflection.Emit;
    using Microsoft.AspNetCore.Identity.EntityFrameworkCore;

    namespace Animals.Infrastructure
    {
        11 references
        public class AnimalContext : IdentityDbContext<AppUser>
        {
        }
    }
}
```

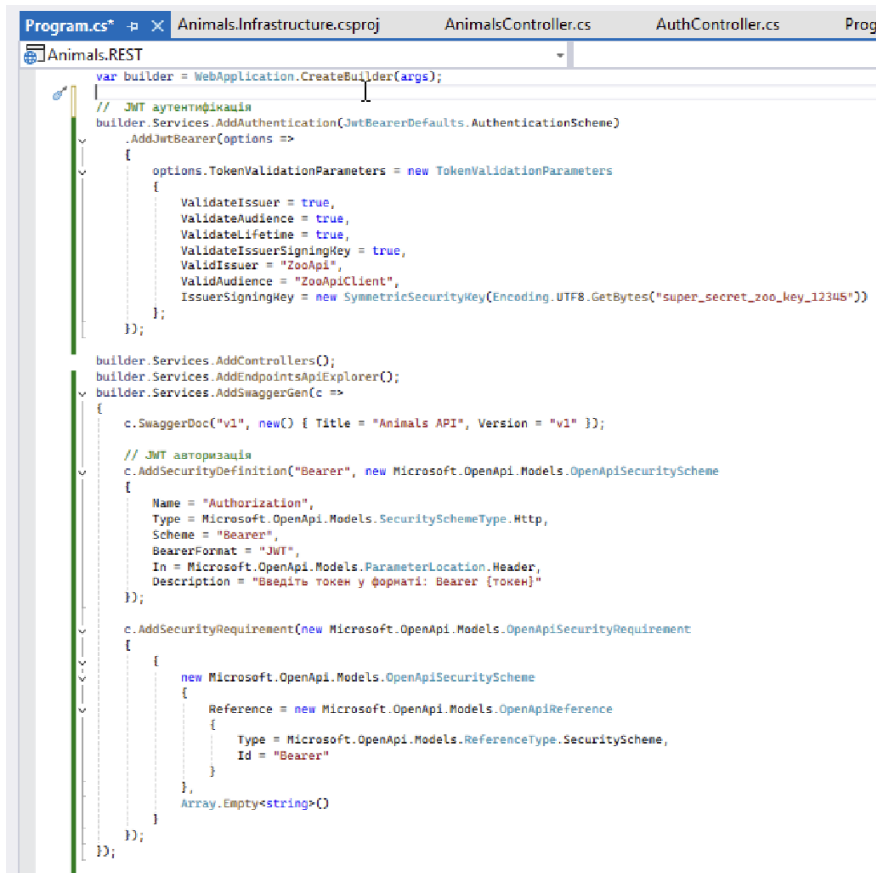
3. Модифікуйте проєкт `{Назва тематики}.REST`, щоб він виставляв на зовні вбудовані у ASP.NET Core Identity кінцеві точки для реєстрації та аутентифікації, при цьому аутентифікація повинна створювати Bearer або JWT токени. Можете спиратися на офіційний туторіал від Microsoft.

Встановлюємо через консоль JWT-пакет



```
PM> Install-Package Microsoft.AspNetCore.Authentication.JwtBearer -Version 8.0.4
Restoring packages for C:\Users\Misha\Desktop\Animals.ConsoleApp\Animals.ConsoleApp.csproj...
  CACHE https://d1w7102yg62uho.cloudfront.net/v3/vulnerabilities/index.json
  CACHE https://api.nuget.org/v3-vulnerabilities/2025.05.23.23.37.46/vulnerability.base.json
  CACHE https://api.nuget.org/v3-vulnerabilities/2025.05.23.23.37.46/2025.05.26.05.37.51/vulnerability.update.json
Installing NuGet package Microsoft.AspNetCore.Authentication.JwtBearer 8.0.4.
Writing assets file to disk. Path: C:\Users\Misha\Desktop\Animals.ConsoleApp\obj\project.assets.json
Restored C:\Users\Misha\Desktop\Animals.ConsoleApp\Animals.ConsoleApp.csproj (in 176 ms).
Successfully installed 'Microsoft.AspNetCore.Authentication.JwtBearer 8.0.4' to Animals.ConsoleApp
Successfully installed 'Microsoft.IdentityModel.Abstractions 7.1.2' to Animals.ConsoleApp
Successfully installed 'Microsoft.IdentityModel.JsonWebTokens 7.1.2' to Animals.ConsoleApp
Successfully installed 'Microsoft.IdentityModel.Logging 7.1.2' to Animals.ConsoleApp
Successfully installed 'Microsoft.IdentityModel.Protocols 7.1.2' to Animals.ConsoleApp
Successfully installed 'Microsoft.IdentityModel.Protocols.OpenIdConnect 7.1.2' to Animals.ConsoleApp
Successfully installed 'Microsoft.IdentityModel.Tokens 7.1.2' to Animals.ConsoleApp
Successfully installed 'System.IdentityModel.Tokens.Jwt 7.1.2' to Animals.ConsoleApp
Executing nuget actions took 447 ms
Time Elapsed: 00:00:01.0380060
PM>
```

В проєкті Animal.REST у файлі Program.cs додаємо цей код, який відповідає за авторизацію та аунтифікацію



```
var builder = WebApplication.CreateBuilder(args);

// JWT аутентифікація
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = "ZooApi",
            ValidAudience = "ZooApiClient",
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("super_secret_zoo_key_12345"))
        };
    });

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new() { Title = "Animals API", Version = "v1" });

    // JWT авторизація
    c.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = Microsoft.OpenApi.Models.ParameterLocation.Header,
        Description = "Введіть токен у форматі: Bearer {token}"
    });

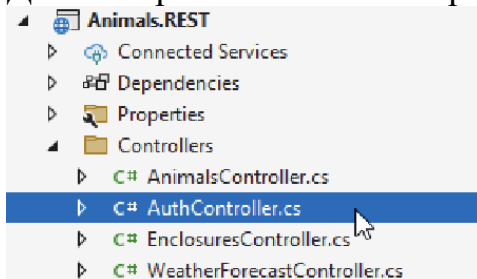
    c.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
    {
        {
            new Microsoft.OpenApi.Models.OpenApiSecurityScheme
            {
                Reference = new Microsoft.OpenApi.Models.OpenApiReference
                {
                    Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

Також вписуємо у файл json ось цей код для JWT



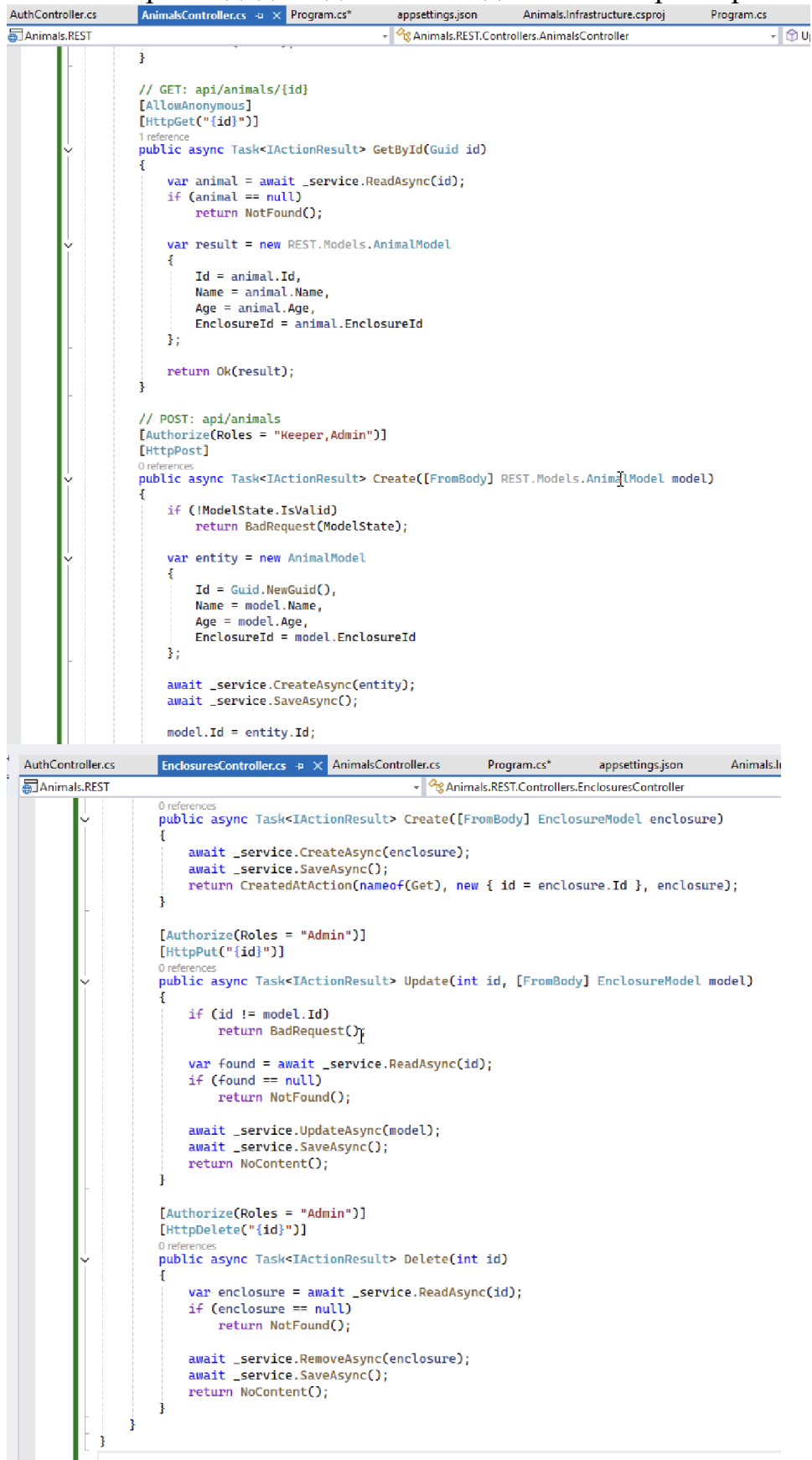
```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=animals.db"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "Jwt": {
    "Key": "super_secret_zoo_key_123456789_ABCDEF_XYZ",
    "Issuer": "ZooApi",
    "Audience": "ZooApiClient"
  }
}
```

Далі створюємо новий контроллер AuthController.cs у папці Controllers



в наступних завданнях його заповнимо

Також потрібно додати деякі зміни до обох контроллерів



The image shows two screenshots of a Visual Studio code editor. The top screenshot displays the `AnimalsController.cs` file. It contains two methods: `GetById` and `Create`. The `GetById` method is a GET endpoint for `api/animals/{id}` that returns an `AnimalModel`. The `Create` method is a POST endpoint for `api/animals` that creates a new animal and returns the created entity. The bottom screenshot displays the `EnclosuresController.cs` file. It contains three methods: `Create`, `Update`, and `Delete`. The `Create` method is a POST endpoint for `api/enclosures` that creates a new enclosure. The `Update` method is a PUT endpoint for `api/enclosures/{id}` that updates an enclosure. The `Delete` method is a DELETE endpoint for `api/enclosures/{id}` that deletes an enclosure. The code is written in C# and uses ASP.NET Core conventions.

```
// GET: api/animals/{id}
[AllowAnonymous]
[HttpGet("{id}")]
public async Task<IActionResult> GetById(Guid id)
{
    var animal = await _service.ReadAsync(id);
    if (animal == null)
        return NotFound();

    var result = new REST.Models.AnimalModel
    {
        Id = animal.Id,
        Name = animal.Name,
        Age = animal.Age,
        EnclosureId = animal.EnclosureId
    };

    return Ok(result);
}

// POST: api/animals
[Authorize(Roles = "Keeper,Admin")]
[HttpPost]
public async Task<IActionResult> Create([FromBody] REST.Models.AnimalModel model)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    var entity = new AnimalModel
    {
        Id = Guid.NewGuid(),
        Name = model.Name,
        Age = model.Age,
        EnclosureId = model.EnclosureId
    };

    await _service.CreateAsync(entity);
    await _service.SaveAsync();

    model.Id = entity.Id;
}
```

```
public async Task<IActionResult> Create([FromBody] EnclosureModel enclosure)
{
    await _service.CreateAsync(enclosure);
    await _service.SaveAsync();
    return CreatedAtAction(nameof(Get), new { id = enclosure.Id }, enclosure);
}

[Authorize(Roles = "Admin")]
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, [FromBody] EnclosureModel model)
{
    if (id != model.Id)
        return BadRequest();

    var found = await _service.ReadAsync(id);
    if (found == null)
        return NotFound();

    await _service.UpdateAsync(model);
    await _service.SaveAsync();
    return NoContent();
}

[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var enclosure = await _service.ReadAsync(id);
    if (enclosure == null)
        return NotFound();

    await _service.RemoveAsync(enclosure);
    await _service.SaveAsync();
    return NoContent();
}
```

Тут ми показували, хто має доступ до розділів.

4. Модифікуйте створений у четвертій лабораторній REST API, щоб кінцеві точки використовували авторизацію, та надавали доступ відповідно до токenu із заголовку HTTP-запиту Authorization. Зверніть увагу, щоб авторизація використовувалася лише в необхідних кінцевих точках, наприклад у REST API бібліотеки, перегляд книг не потребує авторизації, але бронювання книги потребує.

Тут демонструю повний код контролера AuthController.cs, в якому містяться пункти з 4 і 5 завдання лабораторної. Тут реалізовано авторизації, токени, дозволи на дії певними користувачами та розподілення ролей



```
using Animals.Infrastructure.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace Animals.REST.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class AuthController : ControllerBase
    {
        private readonly UserManager<AppUser> _userManager;
        private readonly SignInManager<AppUser> _signInManager;
        private readonly IConfiguration _configuration;

        public AuthController(UserManager<AppUser> userManager,
            SignInManager<AppUser> signInManager,
            IConfiguration configuration)
        {
            _userManager = userManager;
            _signInManager = signInManager;
            _configuration = configuration;
        }

        [HttpPost("register")]
        public async Task<IActionResult> Register([FromBody] RegisterDto model)
        {
            var user = new AppUser { UserName = model.UserName, Email = model.Email };
            var result = await _userManager.CreateAsync(user, model.Password);

            if (result.Succeeded)
            {
                await _userManager.AddToRoleAsync(user, "User");
                return Ok("Користувача записано");
            }

            return BadRequest(result.Errors);
        }

        [HttpPost("login")]
        public async Task<IActionResult> Login([FromBody] LoginDto model)
        {
            var user = await _userManager.FindByNameAsync(model.UserName);
            if (user == null) return Unauthorized("Користувача не знайдено");

            var result = await _signInManager.CheckPasswordSignInAsync(user, model.Password, false);
            if (!result.Succeeded) return Unauthorized("Невірний пароль");

            var token = GenerateJwtToken(user);
            return Ok(new { token });
        }

        private string GenerateJwtToken(AppUser user)
        {
            var claims = new[]
            {
                new Claim(ClaimTypes.Name, user.UserName),
                new Claim(ClaimTypes.NameIdentifier, user.Id)
            };

            var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));
            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

            var token = new JwtSecurityToken(
                issuer: _configuration["Jwt:Issuer"],
                audience: _configuration["Jwt:Audience"],
                claims: claims,
                expires: DateTime.UtcNow.AddHours(3),
                signingCredentials: creds
            );

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}
```

```

1 reference
public class RegisterDto
{
    1 reference
    public string UserName { get; set; } = "";
    1 reference
    public string Email { get; set; } = "";
    1 reference
    public string Password { get; set; } = "";
}

1 reference
public class LoginDto
{
    1 reference
    public string UserName { get; set; } = "";
    1 reference
    public string Password { get; set; } = "";
}
}

```

- Додайте для створеного вебдодатку ролі, усього ролей повино бути не менше трьох. Оновіть кінцеві точки REST API, що використовують авторизацію, щоб вони виконувалися залежно від ролі клієнта, наприклад у REST API бібліотеки, забронювати книгу може звичайний користувач, але змінити інформацію про книгу у БД лише бібліотекар.

Реєструємо ролі у файлі Program.cs проекту Animal.REST

```

Program.cs*  appsettings.json  Animals.Infrastructure.csproj  AnimalsController.cs  AuthController.cs  Program.cs
Animals.REST
app.UseAuthentication();
app.UseAuthorization();
1 reference
async Task CreateRolesAsync(WebApplication app)
{
    using var scope = app.Services.CreateScope();
    var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    string[] roles = new[] { "User", "Keeper", "Admin" };

    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
            await roleManager.CreateAsync(new IdentityRole(role));
    }
}
await CreateRolesAsync(app);
1 reference
async Task AssignRolesAsync(WebApplication app)
{
    using var scope = app.Services.CreateScope();
    var userManager = scope.ServiceProvider.GetRequiredService<userManager<AppUser>>();

    var keeper = await userManager.FindByNameAsync("keeper1");
    if (keeper != null)
        await userManager.AddToRoleAsync(keeper, "Keeper");

    var admin = await userManager.FindByNameAsync("admin1");
    if (admin != null)
        await userManager.AddToRoleAsync(admin, "Admin");
}

await AssignRolesAsync(app);
app.Run();

```

Після реалізації кода, робимо міграцію, щоб у моїй базі було створено всі потрібні таблиці.

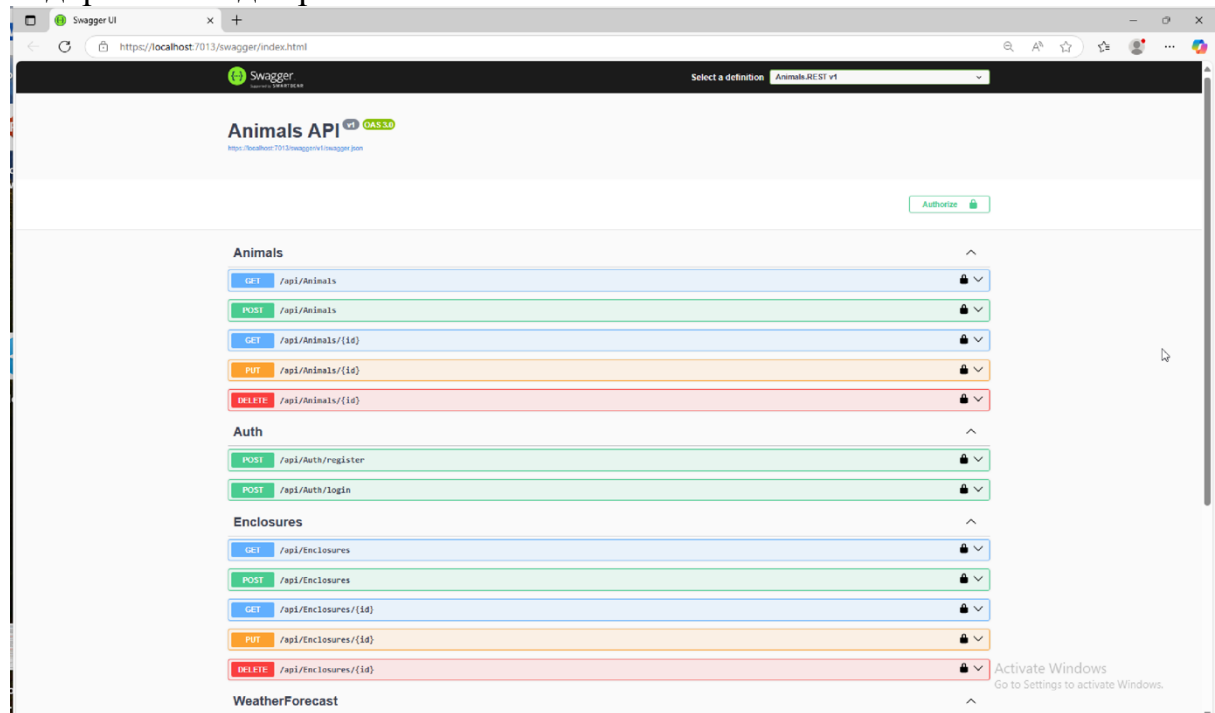
```

PM> Add-Migration AddIdentityTables
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.

Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  CREATE UNIQUE INDEX "UserNameIndex" ON "AspNetUsers" ("NormalizedUserName");
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  INSERT INTO "__EFMigrationsHistory" ("MigrationId", "ProductVersion")
  VALUES ('20250526135124_AddIdentityTables', '8.0.4');
Done.

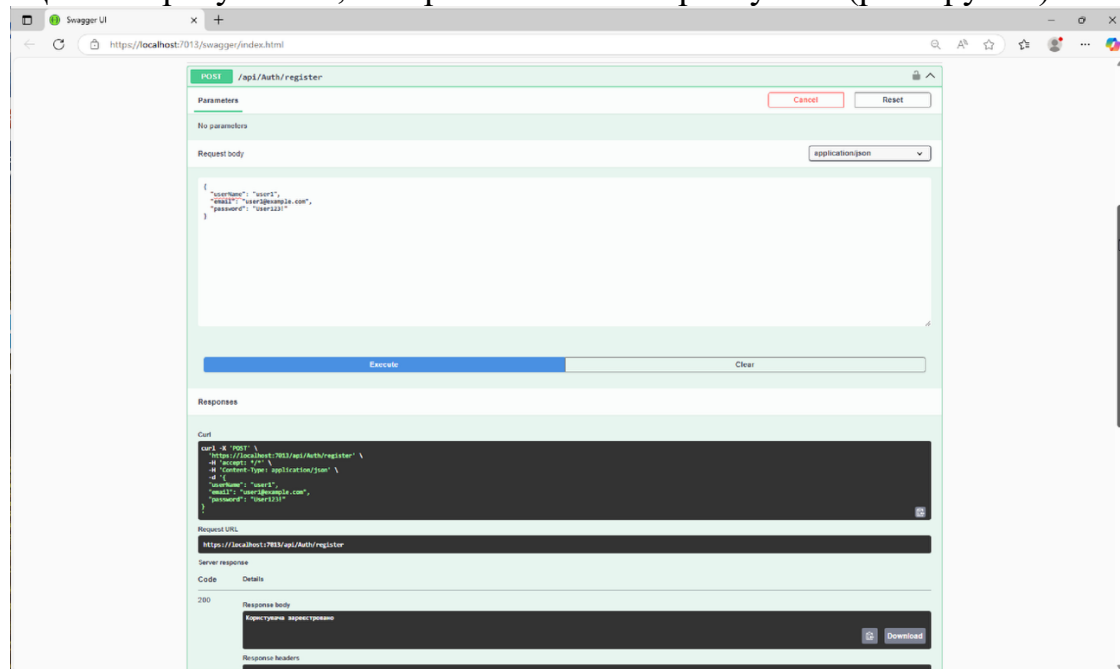
```

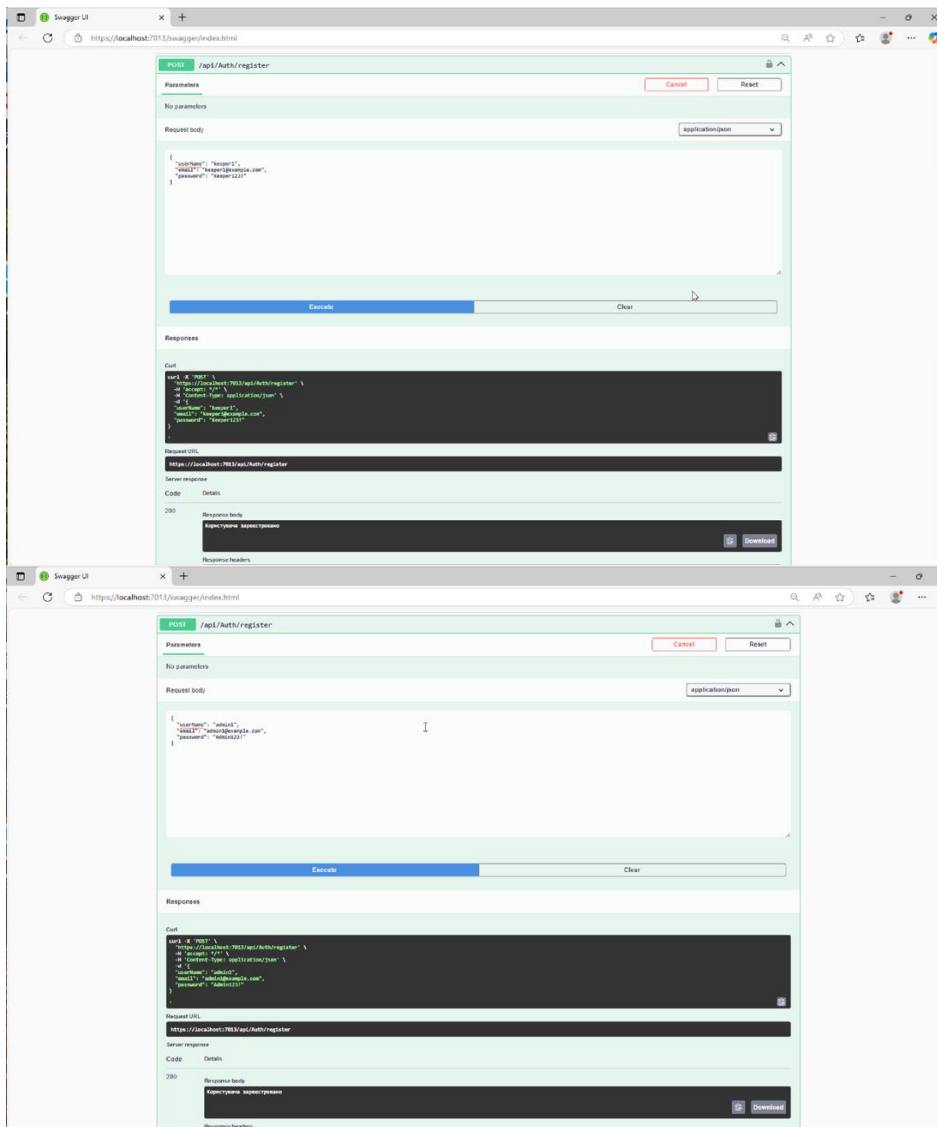
Тестуємо наш застосунок.
Відкритий сваджер:



Оскільки ще не авторизований ніхто, всюди стоять закриті замки.

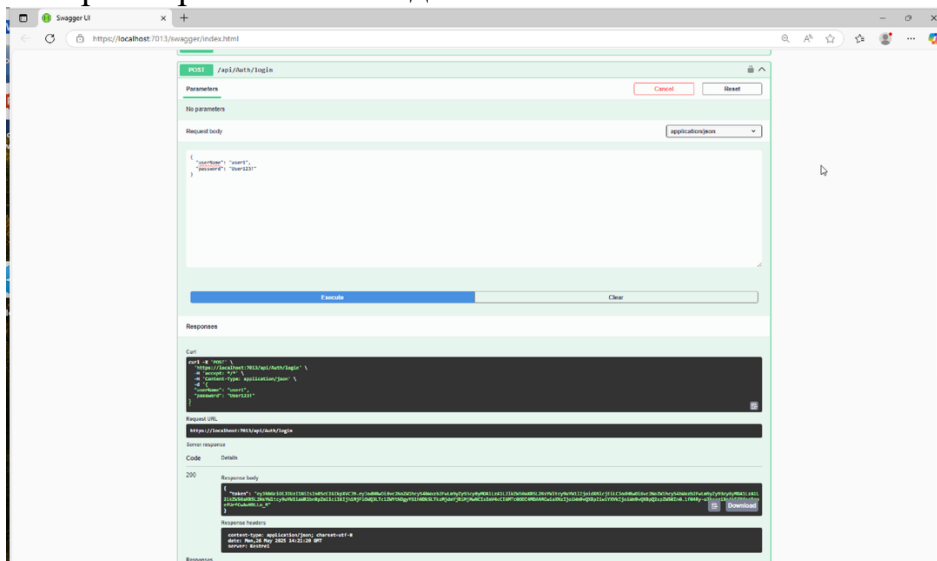
Щоб авторизуватися, створюємо нових користувачів(реєструємо)





Користувачів зареєстровано.

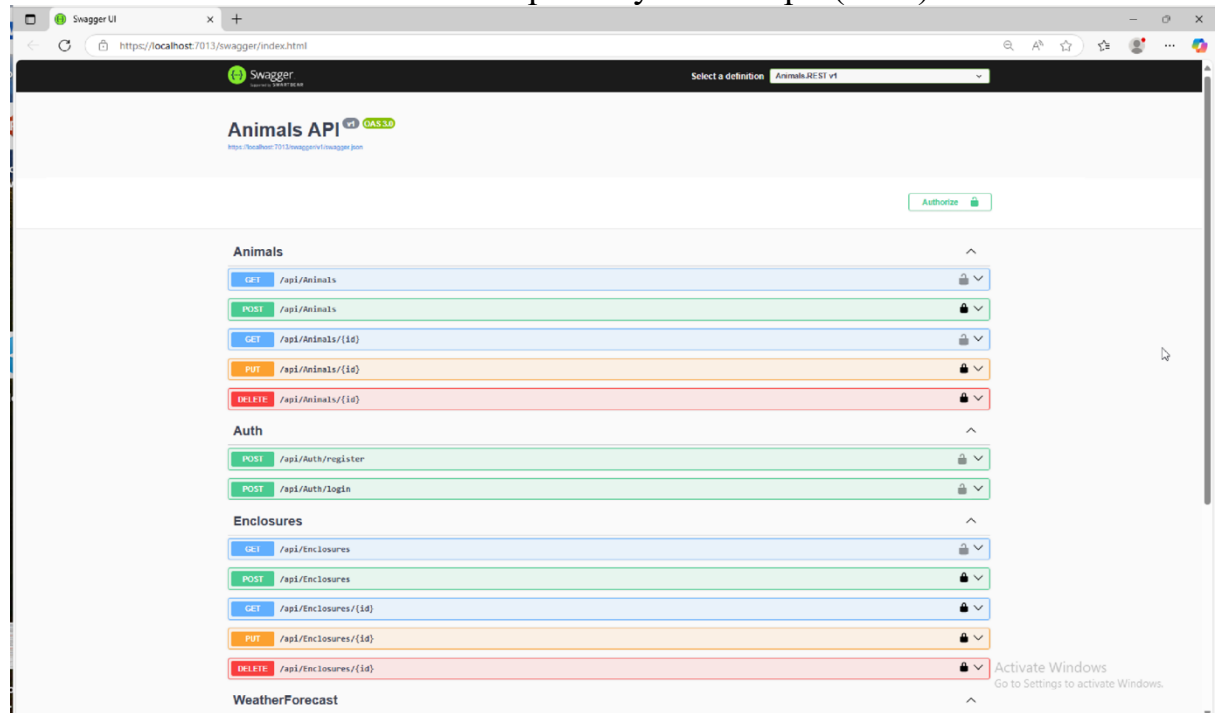
Тепер створюємо токени для них



Тепер авторизуємося під звичайним користувачем і йому можна буде тільки:

В Animals.Controller: подивитися всіх тварин просто та по айді(GET)

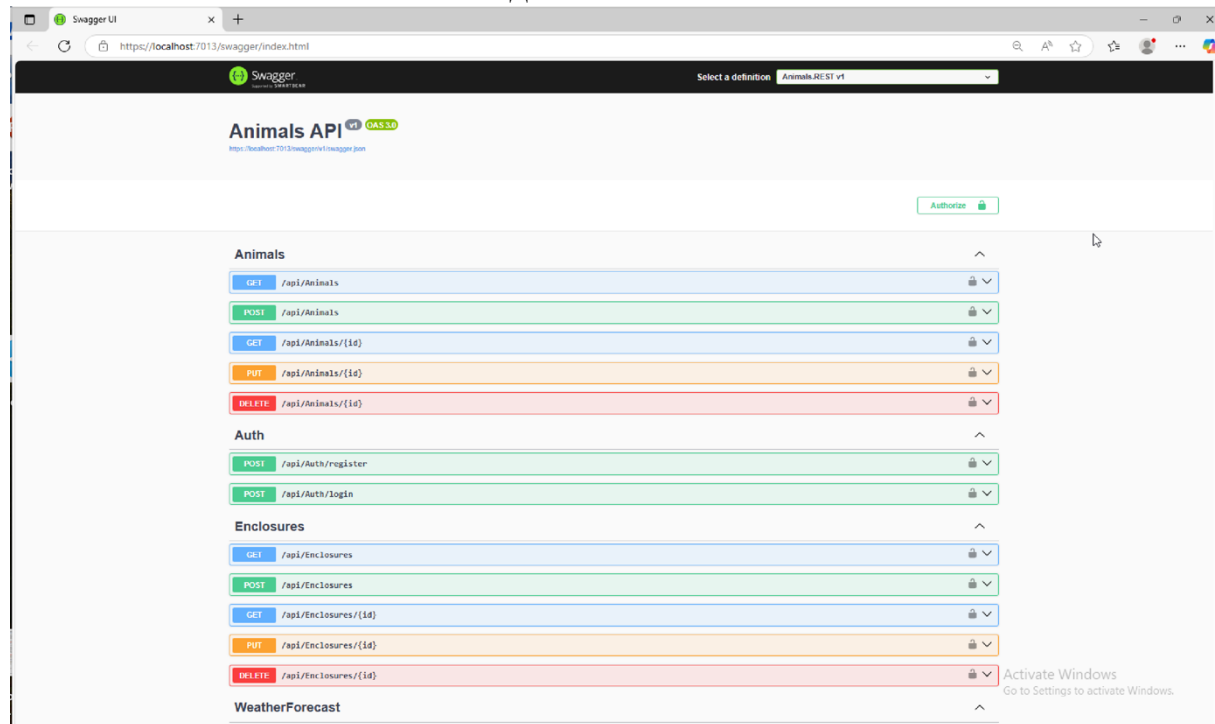
В EnclosuresController: тільки переглянути вольєри (GET)



Тепер авторизуємося під адміном і йому можна буде робити все:

В Animals.Controller: всі методи

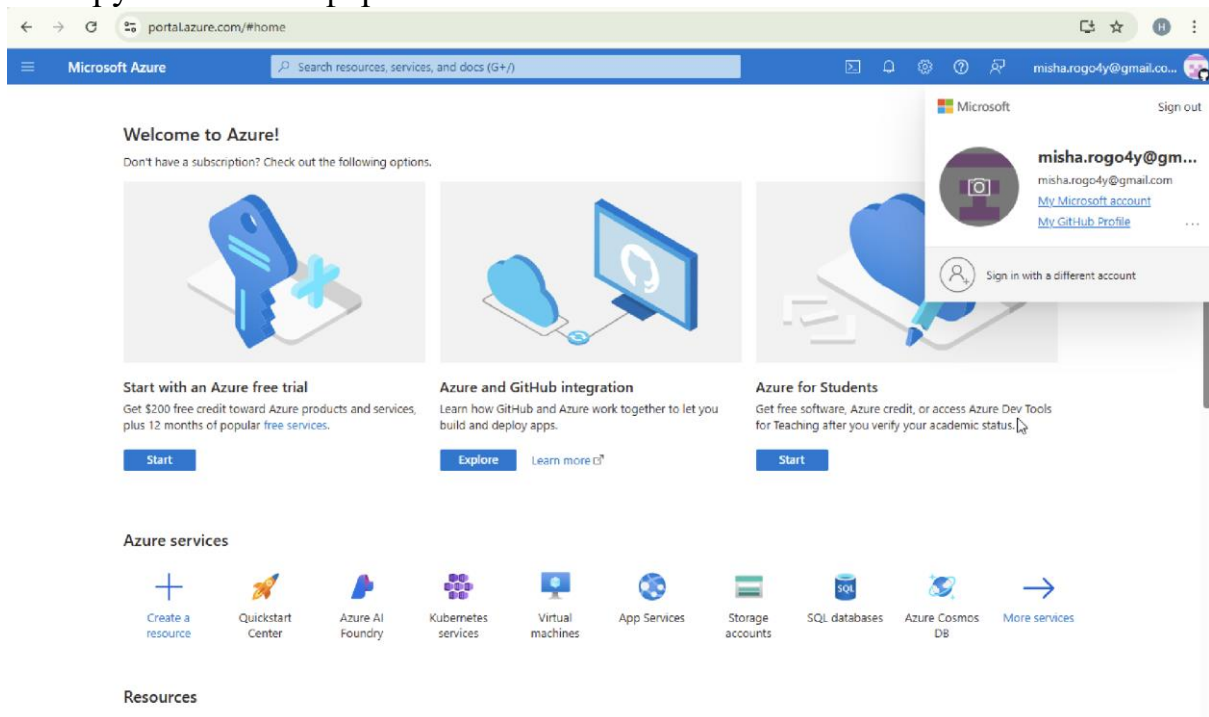
В EnclosuresController: всі методи



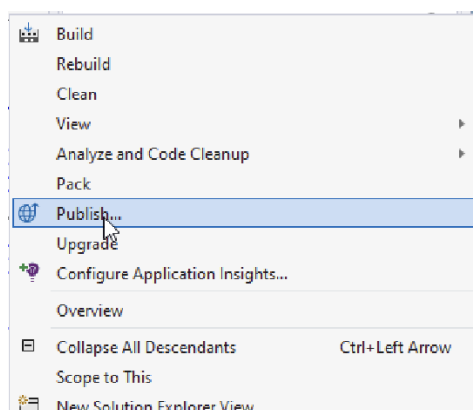
Додаткове завдання:

Розгорніть ваш застосунок на одній із хостингових або хмарних платформ, як Azure, Heroku, AWS, GCP і т.п. У звіті приклади запитів повинні використовувати розгорнутий застосунок. До звіту також додайте скріншоти із розгорнутою інфраструктурою проєкта та посилання на застосунок.

Реєструюся на платформі Azure



Далі виконую ці кроки:



Publish

Where are you publishing today?

Target

**Azure**
Host your application to the Microsoft cloud

**Docker Container Registry**
Publish your application to any supported Container Registry that works with Docker images

**Folder**
Publish your application to a local folder or file share

**Web Server (IIS)**
Publish your application to IIS using Web Deploy or Web Deploy Package

**Import Profile**
Import your publish settings to deploy your app

Back

Next

Finish

Cancel

Publish

Which Azure service would you like to use to host your application?

Target

**Azure Container Apps (Linux)**
Run scalable containerized applications and microservices on a serverless platform in Azure

**Azure App Service (Windows)**
Publish your application code to a managed infrastructure that is easy to scale

**Azure App Service (Linux)**
Publish your application code to a managed infrastructure that is easy to scale

**Azure Container Registry**
Publish your application as a Docker image to Azure Container Registry

**Azure Virtual Machine**
Manage your own infrastructure

Back

Next

Finish

Cancel

Publish

Select existing or create new

Target

**App Service (Windows)**
Create new

Microsoft account

misha.rogod4@gmail.com

Microsoft account

misha.rogod4@gmail.com

Specific target

App Service

API Manager

Name

animals-rest-app

Subscription name

Azure subscription 1

Resource group

AnimalsRG*

New...

Hosting Plan

AnimalsPlan* (West Europe, S1)

New...

Export...

Create

Cancel

Back

Next

Finish

Cancel

Microsoft account
misha.rogoly@gmail.com

Publish

Select existing or create a new Azure App Service

Target

Subscription name
Azure subscription 1

Specific target

animals-rest-app

+ Create new

App Service

AnimalsRG

animals-rest-app

Deployment Slots

API Management

☒ Deploy as ZIP package

Back

Next

Finish

Cancel

Microsoft account
misha.rogoly@gmail.com

Publish

Enable API consumption for teams, customers, and Logic and Power Apps

Target

Subscription name
Azure subscription 1

Specific target

Search

+ Create new

App Service

There are no existing instances available
Create a new instance

API Management

☒ Skip this step

Back

Next

Finish

Cancel

Animals.REST: Publish

AuthController.cs

EnclosuresController.cs

AnimalsController.cs

Program.cs*

appsettings.json

Animals.Infrastructure.csproj

Animals.REST.csproj

Overview

Connected Services

Publish

animals-rest-app - Zip Deploy.pubxml

Azure App Service (Windows)

+ New profile

More actions

Ready to publish.

Settings

Configuration

Release

Target Framework

net5.0

Deployment Mode

Framework-dependent

Target Runtime

Portable

Show all settings

Hosting

Account

misha.rogoly@gmail.com (Microsoft account)

Subscription

88d0c520-984e-4d46-ae55-950e8d08f9e

Resource group

AnimalsRG

Resource name

animals-rest-app

Site

https://animals-rest-app.azurewebsites.net

Service Dependencies

Azure API Management

Microsoft identity platform

SQL Server Database

Connection string name: ConnectionStrings.DefaultConnection

Publish profile creation progress

Target

Specific target

App Service

API Management

Finish

Automatically close when succeeded

Back

Next

Close

Cancel

C# AnimalModel.cs

C# EnclosureModel.cs

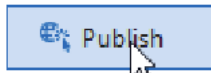
animals.db

Animals.REST.http

appsettings.json

Program.cs

WeatherForecast.cs



Visual Studio interface showing the 'Publish' window for the 'animals-rest-app' project. The window displays the 'Publish' tab with a 'Publish' button and a 'Warming up your site...' status. The 'Settings' section shows the configuration for the application, including the 'Release' configuration, 'net8.0' target framework, 'Framework-dependent' deployment mode, and 'Portable' target runtime. The 'Hosting' section shows the account 'misha.rogo4y@gmail.com (Microsoft account)' and the subscription '8de0e520-984e-4d46-ae55-950e0e0d8f1e'. The 'Output' window at the bottom shows the build and publish logs, indicating that the application was published successfully to the Azure App Service.

Microsoft Azure portal interface showing the 'animals-rest-app' Overview page. The page displays the application's properties, including the Name 'animals-rest-app', Application ID '9d74ca27-44bb-4185-a01a-...', and Object ID '73d0d229-a086-4bcf-bf45-...'. The 'Getting Started' section provides three steps: 1. Assign users and groups, 2. Provision User Accounts, and 3. Conditional Access.